Republic of Yemen

Ministry of Higher Education

Taiz University

Al-Saeed College of Engineering and Information Technology

Technology Engineering Department

Second level

Naqaa Store | Final Project Report

Course: Web Development Fundamentals

Done By :

Elyas Aref Ahmed Abdo(Leader)

Islam Adel Ali Mohammed

Supervisor:

Eng. Abdulsalam Abdulrab

Yemen-Taiz

2025-2026

contents

1. Introduction

English: In the contemporary digital era, web development has transformed from static page design to creating highly interactive, client-side dynamic applications. This academic report presents the technical documentation for "Naqaa Store," a responsive e-commerce web platform engineered for Level 2 Web Technology course. The core objective of this project is to implement core software engineering principles using pure frontend technologies, simulating complex database logic through persistent local storage, and ensuring architecture adaptability across diverse device layouts.

2. Project Description

Naqaa Store is an advanced e-commerce web application dedicated to the cosmetics and skincare industry. The system targets two primary users: the Customer, who can visually browse high-quality products and interact with the store dynamically, and the Admin (Elyas), who possesses exclusive credentials to manage the store's inventory. The scope is rigorously defined to achieve full data autonomy using lightweight client-side technologies, eliminating the dependency on external web servers during evaluation.

3. System Architecture & Folder Structure

To enforce clean code practices, the codebase is organized under a strict, standardized directory tree to facilitate future maintenance and continuous deployment:

3.1. 📂 Naqaa_Store (Root Directory)

3.1.1. 📄 index.html (The public store interface, product grid view, and team presentation tab)

3.1.2. 📄 login.html (Secure portal gating administrative dashboard access)

3.1.3. 📄 view-admin.html (Central administrative control panel displaying inventory tables)

3.1.4. 📄 add.html (Data capture form for adding newly sourced cosmetics)

3.1.5. 📄 edit.html (Data modification form for updating existing stock data)

3.2. 📁 css (Design Assets Directory)

3.2.1. 📄 style.css (Unified style sheet managing global application rules)

)

3.3. 📁 js (Logic & Engine Directory)

3.3.1. 📄 main.js (Pure JavaScript engine driving the entire runtime application)

)

3.4. 📁 images (Media asset repository storing brand logo and product mockups)

3.5. 📁 documents (Academic repository holding technical reports and editable project files)

4. Semantic HTML5 Layout

All user interfaces are written natively using semantic tags defined under the HTML5 specification. This ensures higher markup standards, clean structuring, and search engine optimization (SEO):

4.1. <header> & <nav>: Constructs the sticky application navigation bar housing links and interactive dark mode toggles.

4.2. <main>: Structural wrapper keeping the primary content isolated from layout utilities.

4.3. <section>: Segregates independent UI blocks, such as switching views between the storefront and team documentation.

4.4. <table>: Renders highly structured tabular matrix data for managing products and listing developers.

4.5. <footer>: Closes every page with standard copyright notifications and institution details.

5. Advanced CSS3 & Responsive Design

The UI layer is engineered using advanced styling mechanisms to deliver a high-end luxury aesthetic suitable for a modern cosmetics store:

5.1. CSS Variables (:root): Universally defines the design token system, anchoring the core color palette to a deep royal crimson maroon (--primary-color: #4A0E1B) and soft cosmetic pink (--secondary-color).

5.2. Flexbox Layout Model: Utilizes display: flex; properties to coordinate automatic layout wrapping for product cards and balance space distribution inside navigation headings.

5.3. Media Queries (@media): Leverages breakpoint logic targeted at viewports sizing 768px and below. It automatically shifts horizontal list structures into vertical blocks, enabling full responsiveness on mobile phones and tablets.

6. JavaScript Interactivity

JavaScript operates as the structural brain of the engine, ensuring native interactive features without bloating the application with exterior frameworks:

6.1. Smart Tabs Switcher: Intercepts click events on navigation tabs to instantaneously hide/reveal view states (Store vs. About Us) under a single DOM tree without triggering full-page browser updates.

6.2. Dynamic DOM Manipulation: Employs the document.createElement() API to programmatically parse database nodes and instantly paint rows onto admin lists and generate cards onto public storefront views.

6.3. Event Handling: Employs explicit .addEventListener('submit') methods over input nodes to apply validation scripts and execute preventDefault() to guarantee stable client execution.

7. Client-Side State & LocalStorage

To adhere to the requirement of client-side operations, data layer persistence is managed through native storage drivers:

7.1. Serialization Management: Data records are represented as primitive JavaScript object arrays, translated into string data through JSON.stringify() for disk writing, and re-inflated back via JSON.parse() during application initialization.

7.2. Data Persistence: CRUD mutations persist continuously inside browser partitions. Changes survive browser tab destruction, page reloads, and computer shutdowns.

8. Comprehensive CRUD Operation

A fully functional data cycle (CRUD) has been written into the core engine logic to empower the admin to manipulate stock states dynamically:

8.1. Create: Processes inputs via add.html, generates a unique record reference string via Date.now(), constructs the payload object, and appends it to the disk collection.

8.2. Read: Iterates systematically through array layers to output card blocks for shoppers and structural matrices for managers.

8.3. Update: Passes targeted product index tokens to edit.html, pulls current data strings to prepopulate input boxes, and binds updates to the collection using findIndex().

8.4. Delete: Instantly drops selected items out of storage blocks via an optimized item matching array filter() technique and dynamically refreshes UI blocks without page hard-reloads.

9. Security & Firewall Router

9.1. Security Firewall: An internal middleware engine watches state values during routing changes. If an unauthenticated user copies the admin URL directly, the engine flags the breach, drops execution, and triggers an immediate redirect forcing them back to login.html.

9.2. Credentials Check: Processes incoming inputs in the authentication pane, matching them strictly against the configured admin name (elyas) and access key (24816).

10. Project Development Team & Roles Table

Student Name	Student ID	Assigned Role
Elyas Aref Ahmed Abdo	240102010071	JavaScript Logic, Project Documentation, Technical Report Copywriting & Quality Assurance
Islam Adel Ali Mohammed	240102010126	HTML Structure
Abdullah Rassam Sofian Taher	240102010201	CSS Styling

11. Form Components & Validation

To prevent database entry pollution, explicit verification shields protect input fields:

11.1. Authentication Gate (login.html): Restricts query processing unless input schemas map precisely to authorized variables.

11.2. Inventory Sheets (add.html / edit.html):

11.2.1. Title blocks carry HTML5 required flags to block null item naming.

11.2.2. Valuation cells are restricted to type="number" with a floor cap of min="1", removing the risk of processing zero or negative prices.

11.2.3. Asset fields parse data via type="url" rules, ensuring image resources map to readable format structures.

12. User Interface Layout Architecture

Application uniformity is guaranteed by standard design structures used across all page layouts:

12.1. Global Header Element: Houses the permanent navigation routes, corporate store logo, and the theme switcher.

12.2. Global Footer Element: Placed uniformly at the absolute bottom coordinates, displaying the 2026 project name and engineering credits.

12.3. Navigation Safety Anchors: Every mutation pane provides a Cancel & Return escape button to route users back to workspace tables without forcing manual URL entry errors.

13. Project Timeline(2 Weeks)

Week	Days	Project Phase	Achieved Tasks
Week 1	1-3 Days	Planning & Core Layout	Project scoping and approval. Assembly of semantic HTML5 structural sheets for all pages.
Week 1	4-7 Days	Styling & Responsiveness	Variable configurations and Flexbox architecture. Implementing Media Queries for absolute device cross-compatibility.
Week 2	8-11 Days	JavaScript Engine Sprint	Programming the storage drivers, building full CRUD arrays, and deploying URL protection middleware.
Week 2	12-14 Days	Testing & Submission	Dark Mode validation, input error verification, academic writing review, and preparation for the live demonstration.